

开题答辩演讲稿

开题答辩演讲稿

题目：《基于大模型编码智能体的自进化 eBPF 网络入侵防御系统》

答辩人：李兆曦 指导教师：徐恪教授 清华大学计算机科学与技术系 2026
年 6 月

全文约 3300 字，讲述时长约 13–15 分钟。每节标注【对应幻灯片】与建议用时，
可按答辩时限删减。

开场（封面页，约 0.5 分钟）

各位老师好。我是李兆曦，我的指导教师是徐恪教授。我今天开题的题目是《基于大模型编码智能体的自进化 eBPF 网络入侵防御系统》。下面我从研究背景与动机、研究问题与挑战、相关工作、研究内容与方法、技术路线、创新点，以及工作计划与预期成果几个方面，向各位老师汇报。

一、研究背景与动机（第 4–7 页，约 2.5 分钟）

先说研究背景。

TCP/IP 协议栈是互联网的基石，但它在设计之初对安全性考虑不足，又经过数十年的演进与打补丁，协议实现的内部、以及协议与协议之间，形成了大量复杂的跨层交互。近十年的研究表明，正是这些跨层交互中的语义歧义，持续地催生出一类被称为“链外攻击”、也就是

off-path 攻击的威胁: 攻击者既不需要位于通信路径上, 也不需要窃听报文, 仅凭侧信道, 就能推断出 TCP 连接的序列号等关键状态, 进而劫持、重置, 甚至污染连接。

更值得警惕的是, 这个攻击面在不断“再生”。从 2016 年 challenge-ACK 全局限速被用作侧信道, 到混合 IPID、PMTUD 分片、再到 NAT 与 Wi-Fi 环境下的连接劫持, 新的漏洞一个接一个被发现。近期一项 CCS'25 的研究进一步揭示: 连被普遍信任的路径 MTU 发现机制——也就是 PMTUD, 在公网 Wi-Fi、VPN、5G 这类“IP 地址共享”的场景下, 也会被用来打破 TCP 连接之间的隔离; 平均 220 秒、约七成的成功率, 就能推断出受害连接的序列号, 在 50 个真实网络中, 有 38 个存在脆弱性。

这里有一个我特别想强调的规律: 很多新的攻击面, 恰恰是防御机制自己引入的。challenge-ACK 限速, 本来是为了抵御盲注入, 却成了侧信道; 源端口随机化, 本来是为了增加猜测难度, 却被侧信道反推。这说明, 协议安全不是“修一次就一劳永逸”的静态问题, 而是一场随着部署环境和补丁不断演化的动态博弈。

那么, 现有的防御为什么不够用? 我把它归纳为三点结构性缺陷。第一是**零散**: 各打各的补丁, 缺乏统一框架。第二是**滞后**: 先有攻击曝光, 再被动响应, 从披露到上线的窗口往往很长。第三, 也是最根本的——**依赖人工编写内核代码**: 改动周期长、容易出错, 而且经验只留在个人脑子里, 难以沉淀复用。一句话: 防御的“生产方式”, 仍然是人工的、低速的、难以累积的。

好在, 有两项技术正在走向成熟, 让“自动化防御”成为可能。一是 **eBPF**: 它允许在不修改内核源码、不重启系统的前提下, 把程序安全地注入内核热点路径, 而且内置的 verifier, 能在加载之前对程序做形式化的安全验证。二是**大模型编码智能体**: 像 Claude Code、Cursor、Codex、Devin, 以及开源的 SWE-agent 等, 已经能够在仓库级任务上自主地编写、运行、调试代码。把这两者结合起来, “让智能体持续维护内核防御”这件事, 就从设想走向了可行。

二、研究问题与挑战（第 9–10 页，约 1.5 分钟）

基于以上背景, 我的**核心研究问题**是: 能否让大模型编码智能体, 自主维护一套运行在内核的 eBPF 防御库, 使它持续进化, 从而应对不断涌现的协议攻击?

这里请各位老师注意, 我的目标不是“一次性合成一个防御程序”, 而是构建一个**长期自主运转、能力单调累积**的防御系统。

要实现这个目标, 主要有三项关键挑战。第一是**可靠性**: 智能体生成的内核代码一旦存在缺陷, 可能导致网络中断, 或误伤正常流量, 因此必须做到“先验证安全、再上线, 异常可回滚”。第二是**生成质量**: 智能体生成的 eBPF 必须能够编译通过、通过内核 verifier, 并且确实拦

截攻击而不误伤正常流量——这直接决定系统是否可用。第三是**评测**：如何验证系统确有成效——不仅对已知攻击有效，对未知的新攻击亦能防御，且性能开销在可接受范围内。

三、相关工作与对比基线（第 12–14 页，约 1.5 分钟）

接下来是相关工作。

在协议攻防方面，学界在 off-path 攻击上有系列积累，检测方面有 SCENT、SCAD 等方法；但整体上，防御仍然零散、滞后。在 eBPF 运行时防御方面，工业界有 Katran、Cilium、Falco、Tetragon，学术界有围绕 verifier 的 PREVAIL、K2——它们能力很强，但策略几乎都由人工编写。在大模型用于安全方面，进攻侧有 PentestGPT、Big Sleep；与我最直接相关的是 Kgent 和 KEN，它用大模型从自然语言合成 eBPF，但它是一次性合成，并不是长期自主维护。在方法论上，我借鉴了“非梯度演化”的思路，以及 FunSearch、AlphaEvolve 这类用大模型做程序演化搜索的工作，还有 Darwin-Gödel Machine 这类自进化智能体的工作。

我用一张表来概括本文的定位：相比 Kgent 的“一次性合成”、Tetragon 和 Falco 的“人工策略”、以及小模型的“权重学习”，本文要做的，是一套长期自主维护、能自我进化、并且知识以“可验证代码”形式沉淀的防御库。

四、研究内容与方法：防御演化（第 16–18 页，约 2.5 分钟）

下面是本文的核心，我把这一范式称为“**防御演化**”，英文是 Defensive Evolution。

它的定义是：防御能力不再以神经网络的权重存续，而是以“经过内核验证器证明安全的 eBPF 代码”作为知识载体，由编码智能体以**非梯度**的方式持续地衍生、修正与沉淀，随着攻击的演进而自我增益。以可验证代码作为载体，带来几方面好处：容量无界、抗灾难性遗忘、可解释、能线速执行，而且可以形式化验证。

本文的研究内容分为两部分。**第一部分**是构建一套完整的专用 harness：将“由编码智能体生成 eBPF 防御并保证其可用、安全”的全过程，封装为一套领域专用 harness——涵盖防御程序的生成、编译、内核 verifier 校验、沙箱测试、加载与运行时反馈，并解决生成质量（可编译、通过 verifier、不误伤正常流量）与运行安全（异常时自动回滚）。**第二部分**是面向真实协议攻击的防御与评测：以 PMTUD、NAT、IPID 等真实攻击构建攻防靶场，由系统自动生成防御，进而度量检测率、误报率与性能开销，并与人工规则及现有工具对比。

这是系统的总体架构，我把它分成三层。最上面是**控制面**，也就是**编码智能体**，包含攻击面探

索、eBPF 合成、以及反思自修正三个部分, 它们都通过专用 harness 来调用工具; 中间是**可信验证面**, 候选程序依次经过编译、内核 verifier、隔离沙箱, 到自动回滚; 最下面是**内核数据面**, 也就是 XDP 上的 tail-call 防御链, 负责线速地放行或丢弃, 并采集遥测。左边的攻防靶场为系统提供攻击样本, 右边的防御库沉淀已验证的防御、并供智能体采样。主流程自上而下——候选 eBPF 下到验证面, 验证通过后热加载到数据面; 而运行时的遥测和验证报错, 会回灌到控制面, 驱动下一轮迭代。

五、技术路线: 防御演化闭环 (第 20–23 页, 约 2 分钟)

具体的技术路线, 是一个五阶段的开放式闭环。

第一步, **攻击面探索**: 以新颖性与覆盖度作为信号, 优先去探查现有防御覆盖不到的方向。第二步, **防御合成**: 让大模型作为“进化算子”, 生成或改写 eBPF 程序。第三步, **可信验证**: 先编译、再过内核 verifier; 如果不通过, 就把报错喂回去, 让它自我修正; 通过之后, 还要在隔离沙箱里, 用攻击流量做对抗评估。第四步, **部署与反馈**: 用 tail-call 把防御热加载到 XDP, 并采集遥测。第五步, **档案更新**: 维护一个多样化的防御库。之后, 运行时的反馈再回灌到第一步, 如此持续演化。每一轮迭代, 都把新发现的攻击面, 转化成一段经过 verifier 背书的 eBPF 防御, 沉淀进库。

这里我要专门讲一下**可信保障**, 因为这是“让智能体写内核代码”能否成立的关键。需要说明的是, 静态沙箱、模板封装、编译器零容忍、内核 verifier 这些, 都是**现成的基础设施**; 本文并不重新发明它们, 而是**把它们组织进智能体的闭环**——用编译和 verifier 的报错作为反馈, 驱动智能体自我修正; 用隔离沙箱做攻击重放把关; 任何一层不通过、或者运行时出现异常, 就自动回滚。正是这套组织方式, 让“智能体自动写内核代码”变得可靠。

另外, 这项工作并不是从零开始。已有的开源系统 PacketScope 中的 Guarder 模块, 已经具备 XDP 数据面、一键编译、规则热加载, 还有一个 Agent 可编程 eBPF 的原型和配套测试; 这为我快速搭建闭环, 提供了现成的底座。

最后说明**评测方法**, 因为这是衡量“自主防御”成不成立的关键。核心是“攻防协同演化”: 我会把攻击集分成训练种子和**预留的未知攻击**; 评测时主要看四件事——第一, 对已知攻击的检出率和误报率; 第二, 对预留的未知攻击, 在没有人工补丁介入的情况下, 防御覆盖随迭代轮次如何提升, 这衡量的就是“自主进化”的能力; 第三, 开启防御以后, XDP 数据面的吞吐损失和时延开销; 第四, 达到某个防御水平所需要的人工介入量, 也就是自动化程度。对比上, 我会和一次性合成的 Kgent、人工写策略的 Tetragon 和 Falco 做比较, 并且做消融实验——分别去掉探索、去掉档案、去掉自修正, 来验证每个部件的贡献。

六、工作计划与预期成果（约 1 分钟）

工作计划上：我计划今年下半年，完成范式的形式化与系统框架原型；年底前，完成闭环原型和初步实验；明年上半年，完成评测与对比实验，并通过中期检查；随后撰写论文、投稿，完成预答辩与送审；到明年六月答辩。

预期成果包括：一篇硕士学位论文；开源自进化 eBPF 防御系统、专用 harness，以及配套的攻防评测基准；此外，如果在研究中发现并自动缓解了新的协议攻击，我会向 IETF、Linux 社区做负责任披露。

结束语（致谢页，约 0.5 分钟）

以上就是我的开题报告。简单总结一下：我希望把自己在协议攻击上的积累，反向用于防御，让编码智能体以“防御演化”的方式，自主维护一套运行在内核、能够持续进化的 eBPF 防御库。恳请各位老师批评指正，谢谢！

附：答辩提问预案（备用，不在正式讲述中）

Q1: 让 AI 写内核代码，安全性如何保证？ 关键在于多层、且不可绕过的约束：静态沙箱、模板封装、编译器零容忍，最后是内核 verifier——它会从数学上拒绝越界、无界循环、非法内存访问的程序；再叠加隔离沙箱对抗评估与运行时自动回滚。即使大模型被对抗性提示，不安全的程序也无法加载到内核。

Q2: 为什么不直接训练一个模型，或直接用 Claude Code 这类现成工具？ 训练模型面临容量受限、灾难性遗忘、且不可形式化验证的问题；而通用编码智能体并不内置内核 eBPF 的领域工具与反馈，直接套用难以胜任。本文以“可验证代码”为知识载体，并设计专用 harness，把通用能力专门化到内核安全任务。

Q3: 如何评测“对未知攻击”的防御能力？ 做法是：把攻击集分成两部分——一部分作“训练种子”，让系统在自我迭代中使用；另一部分预先留出、整个迭代过程都不让系统接触，当作“没见过的新攻击”。评测时度量系统在无人工补丁介入下、对这批未见攻击的防御覆盖随迭代怎么提升，同时报告误报率与性能开销。

Q4: 与已有协议攻击工作是什么关系？ 已有的 off-path 协议攻击工作 (PMTUD、NAT、IPID

等), 既是本系统要防御的真实威胁来源, 也是攻防靶场中红队的高质量种子; 把对协议攻击的深入理解反向用于防御, 是本课题的立足点。

Q5: 工作量与可行性? 工程上以已开源的 PacketScope/Guarder 为底座 (已有 XDP 数据面、一键编译、热加载与 Agent 可编程 eBPF 原型); 方法论上, FunSearch、AlphaEvolve、Darwin-Gödel Machine 等已验证 “大模型进化式程序搜索” 在系统级代码上的可行性。